

Module 3

UART — Protocol + RTL + Basic Testbench

Learning objectives

- UART 8N1: Start bit (0), 8 data bits (LSB first), stop bit (1); idle high.
- Baud rate: Role of a baud generator (clock divider → baud_tick).
- RTL architecture: TX (parallel→serial, shift per baud_tick), RX (start detect, sample, data_valid),...
- Basic TB: Loopback, directed stimulus, and checking without UVM.

Prerequisites

- Module 1 (spec → RTL) and Module 2 (basic TB → UVM, toolchain).
- Verilator, Make, C++ compiler. No UVM required for the uart_baseline example.

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["UART Protocol (8N1)"]

T2["Hands-on testbench (uart_bas)"]

T1 --> T2

T2 --> N["UART UVM+SV"]

Understand the UART protocol (8N1, baud), translate it to RTL (TX, RX, baud gen), and verify with a **basic** (non-UVM) directed testbench.

Overview

- Module 3 is the first **protocol** module:

How to learn this module

Suggested learning path

- Read the detailed UART learning guide:
UART_LEARNING_GUIDE.md — what UART is, what kind of protocol...
- Read the protocols + UVM overview:
LEARNING_GUIDE_PROTOCOLS_AND_UVM.md — Part B § 4.
When to Use Ba...

Design architecture

RTL / block architecture

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart TB

CPP["sim_main.cpp\nclk, rst_n"]

BG["baud_gen\nDIVIDER → baud_tick"]

TX["uart_tx\nparallel → serial"]

RX["uart_rx\nserial → parallel"]

CPP --> BG

Module 3: DUT / block hierarchy (see module3/examples/).

Verification environment architecture

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart LR

STIM["initial block\nstart, data_in\n0x55, 0xAA"] --> TX["uart_tx"]

TX -->|loopback| RX["uart_rx"]

RX --> CHK["wait data_valid\ncompare data_out"]

CHK --> PASS["PASS / \$error"]

Module 3: UVM layers, agents, and checkers for this phase.

1. Block hierarchy

- top_uart_baseline → `baud\_gen`, `uart\_tx`,
`uart\_rx`; sim_main.cpp drives `clk`/`rst\_n` only.
- Loopback: `assign rx = tx` — one wire connects transmitter to receiver.

2. RTL blocks

- baud_gen: ``DIVIDER`` → ``baud_tick`` every N clocks; shared by TX and RX.
- uart_tx: parallel-in / serial-out; start + 8 data (LSB first) + stop; ``busy`` during frame.
- uart_rx: start detect, sample per ``baud_tick``, ``data_valid`` + optional ``framing_error``.

3. Timing and clocking

- Single ``clk`` domain; async ``rst_n``; all bit times referenced to ``baud_tick``.

Verification & testing methods

Testing and closure flow

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

STIM["directed bytes"] --> TX["uart_tx"]

TX --> RX["uart_rx loopback"]

RX --> CHK["data_out check"]

CHK --> PASS["baseline PASS"]

Module 3: how tests, checks, and metrics close verification goals.

1. Verification goals

- Prove RTL implements UART 8N1 before UVM (Module 4).
- Happy-path loopback: bytes sent on TX appear correctly on RX.

2. Baseline directed test

- Stimulus: ``initial`` block — ``start``, ``data_in`` (0x55, 0xAA).
- Check: ``wait(data_valid)``; compare ``data_out``; ``$error`` / ``$display``.
- No UVM: pin wiggling only — fastest way to learn the protocol on real RTL.

3. Coverage gaps (defer to Module 4)

- Random baud, framing errors, `busy` back-to-back, functional coverage.
- Map checks to `[UART_LEARNING_GUIDE.md](UART_LEARNING_GUIDE.md)` spec bullets.

Syllabus topics

1. UART Protocol (8N1) (1/4)

- Signals: Serial line (tx/rx), idle high; system clock; baud_tick (one pulse per bit time).
- Frame: 1 start bit (0), 8 data bits (LSB first), 1 stop bit (1).
- Timing: One bit per baud_tick; baud_tick derived from system clock via a divider (e.g. $DIVIDER = clk_hz / baud$).
- Framing: `idle=1` → `start=0` → 8 data bits (LSB first) → `stop=1`.
- Baud math: $DIVIDER = round(clk_hz / baud)$.
Example: 50 MHz clock, 115200 baud → divider ≈ 434 .
- Reset state: Line idles high, TX not busy, RX clears internal shift register, `data\_valid` low.

1. UART Protocol (8N1) (2/4)

- Errors: ``framing_error`` when stop bit is not high at expected sample; optional ``parity_error`` if pa...
- Inputs: ``clk``, ``rst_n``, ``baud_tick``, ``start``,
``data_in[7:0]``.
- Outputs: ``tx``, ``busy``.
- Behavior: On ``start``, latch ``data_in``, drive start bit low, then shift out bits [0:7] each ``baud_ti...`
- Edge cases: Ignore ``start`` while ``busy`` (or queue it in a FIFO if you extend the design).
- Inputs: ``clk``, ``rst_n``, ``baud_tick``, ``rx``.

1. UART Protocol (8N1) (3/4)

- Outputs: ``data_out[7:0]``, ``data_valid`` (pulse), ``framing_error``.
- Robustness options: Add metastability sync on ``rx``; add oversampling + majority vote; add ``rx_ready``...
- Inputs: ``clk``, ``rst_n``, ``divisor`` (constant parameter in the baseline).
- Output: ``baud_tick`` high for one cycle every ``divisor`` clocks.
- For synthesis, keep the counter simple (``divisor-1`` down-counter or up-counter compare). For sim, y...
- Data bits: 5–9 data bits are common.

1. UART Protocol (8N1) (4/4)

- Parity: Even/odd/mark/space; adds a parity bit before stop.
- Stop bits: 1 or 2 stop bits.
- Flow control: RTS/CTS hardware pins (outside the serial line) to throttle traffic.
- The baseline RTL assumes 8N1 with no parity and no flow control.

2. Hands-on testbench (uart_baseline)

- Loopback: Connect TX output to RX input; send bytes from TX, check they appear on RX.
- Directed test: Reset release, send 0x55 and 0xAA, wait for ``data_valid``, check ``data_out``. Implemen...
- Self-check flow: ``wait(rst_n)`` → pulse ``start`` / ``data_in`` → ``wait(!busy)`` → ``wait(data_valid)`` → com...

Hands-on examples

Module 3 self-check

```
$ ./scripts/module3.sh --help
```

Terminal: module3_check

```

[0:36m=====
[0:36mModule 3: Demo commands (UART baseline)
[0:36m=====
```

From repo root:

```
# 1) Environment sanity:
verilator --version
make --version
```

```
# 2) Run the UART baseline example (loopback, basic TB, no UVM):
cd module3/examples/uart_baseline
make run
```

```
[1:33m[INFO] See module3/EXAMPLES.md and module3/examples/uart_baseline/README.md for more.
```

Exercise scaffold

```
./scripts/module3.sh --scaffold
```

Example 1: UART baseline

- ``dut/`: `uart_tx.v`, `uart_rx.v`, `baud_gen.v``
- ``top_uart_baseline.sv``: Top with loopback; directed stimulus and checks in initial blocks.
- ``sim_main.cpp``: C++ harness (clock, reset); simulation runs until ``$finish``.

Demo: UART baseline

```
$ ./scripts/module3.sh --run
```

Terminal: uart_baseline

```
[0:36m=====
[0:36mModule 3: Run uart_baseline example (logging to /mnt/d/proj/universal-verification-methodolog
[0:36m=====

[Thu May 28 15:27:13 CST 2026] Running make run
Compiling UART baseline with Verilator...
verilator -sv --timing -Wall -Wno-fatal -Wno-TIMESCALEMOD -Wno-WIDTHTRUNC --cc --exe top_uart_basei
/bin/sh: 1: verilator: not found
make: *** [Makefile:23: compile] Error 127
```

Practice & assessment

What you should know

- Describe UART 8N1 frame and baud timing.
- Explain the role of `uart_tx`, `uart_rx`, and `baud_gen`.
- Run the `uart_baseline` example and interpret the loopback test.
- Ready for Module 4: UART UVM+SV verification.

Exercises

- Run `uart_baseline`
- Trace one byte
- Baud and divider
- Optional: Waveforms

Assessment checklist

- Can describe UART 8N1 (start bit, 8 data LSB first, stop bit) and the role of baud_tick.
- Can explain the role of uart_tx, uart_rx, and baud_gen in the RTL.
- Can run uart_baseline and interpret the loopback test result.
- Ready for Module 4: UART UVM+SV verification.

Summary & next steps

- Understand the UART protocol (8N1, baud), translate it to RTL (TX, RX, baud gen), and verify with a...
- Complete module3/CHECKLIST.md
- Review module3/EXAMPLES.md and run each lab
- Next: UART UVM+SV